

Velora - Progress, Decisions and Duck Review Protocol

Version: 1.0 | Initial entry: 2026-10-01 (Asia/Kolkata)

Read with VELORA_MASTER.md and ARCHITECTURE.md.

Intended repository location: docs/SESSION_LOG.md.

1. Current truth

- Owner: Swayam Jain.
- Brand selected: Velora.
- Active product scope: Code 1 post-discharge recovery companion.
- Code 2: future feature inside Velora, not current implementation.
- Code 3: not selected.
- Current stage: documentation / M0.
- Source repository URL/path/branch: not supplied or inspected for this project.
- Implemented features: **none verified**. This is not proof that no code exists elsewhere.
- Tests/build/deployment: not run or verified for an application.
- Clinical partner/procedure: undecided.
- Current next action: connect the actual repository, read its instructions and existing code, then finish M0 decisions and implement the M1 manual vertical slice.

These documents are created deliverables. They do not themselves install repository automation, grant access to the user's laptop, or ensure another AI will load them. Put them in the repository's docs/ folder and reference the bootstrap instruction below from the coding agent's startup instructions.

2. User decisions preserved

Decision	Status
Explore healthcare after rejecting developer-tool ideas	Historical context
Code 1: recovery companion; Code 2: tests/follow-up workflow	Defined
Build Code 1 first, add Code 2 later as a feature	Locked
Use Velora as product name	Locked; availability not verified
Use clinician approval and role/record-based permissions	Baseline
Create three documents to prevent human/AI scope drift	Completed by this document set
Trigger a project audit with the word Duck	Protocol defined below

3. Duck - standing review instruction

When the user says **Duck**, **duck**, or clearly asks for a Duck review:

A. Establish evidence

1. Read the current authoritative versions of all three documents; do not rely on old chat summaries.
2. Read applicable repository instructions (including AGENTS.md when present).
3. Identify repository root, branch, commit, dirty working tree and current milestone. Preserve user changes.
4. Inspect actual source, routes, models/migrations, UI paths, configuration and relevant tests. Do not print secrets, tokens or patient data.
5. Map work to requirement IDs. Distinguish code that exists, code that runs, verified behaviour and unavailable evidence.

If the repository or files are inaccessible, report **BLOCKED / UNVERIFIED**, not ON TRACK. Ask only for the missing repo/path/files needed to inspect; continue any useful document-only review meanwhile. Never claim to have inspected a laptop or repository without access.

B. Compare requirements with behaviour

For the active milestone, classify each relevant requirement:

- VERIFIED: observed implementation plus appropriate evidence.
- IMPLEMENTED / UNVERIFIED: code exists, evidence insufficient.
- PARTIAL: acceptance criteria only partly met.
- NOT STARTED / PLANNED: correctly deferred within the agreed roadmap.
- DRIFT: behaviour contradicts requirements or introduces unapproved scope.
- BLOCKED: access, dependency or a material unresolved decision prevents progress.

Planned later work is not a defect. Focus on current work plus cross-cutting clinical, privacy and authorisation invariants.

C. Act according to the finding

On track: state the evidence and continue the next authorised implementation task. If the request was explicitly review-only, report without changing code.

Ordinary implementation drift: user authorises focused, reversible corrections required to restore agreed behaviour. Choose the smallest appropriate fix, implement, run relevant checks, update documentation/evidence and continue. Do not repeatedly ask permission for normal in-scope fixes.

Critical mismatch: prioritise broken isolation, unauthorised approval, exposed documents, draft-generated reminders, stale medication schedules and data-loss risks. Correct locally within available authority before adding dependent features; never declare readiness while unresolved.

Material product ambiguity: prepare the concrete alternatives and identify the one decision needed. Do not invent clinical instructions, a patient's consent, regulatory approval or a new business scope.

Actions outside automatic correction: no destructive data deletion, irreversible migrations, force-push/history rewriting, paid services, publication/deployment, external messages or production patient-data changes solely because Duck was invoked. Obtain explicit task authorisation where needed and follow runtime approval rules. Never bypass an access or approval block.

Never change the requirements to hide a mismatch, remove tests to hide a failure, auto-promote Code 2, or rewrite unrelated modules to make the project appear more advanced.

D. Verify and close the loop

Run the narrow checks needed for changed behaviour and the relevant gate. Record real commands/results, including failures and tests not run. Recheck the original mismatch after the fix. Update traceability and append a session entry. State remaining risks honestly.

Report template:

```
DUCK REVIEW - <date, branch, commit or unavailable>
Verdict: ON TRACK / DRIFT FIXED / ACTION REQUIRED / BLOCKED
Milestone and scope reviewed:
Evidence inspected:
Requirement findings: ID | expected | actual | status | evidence
Corrections made:
Verification: commands/results; limitations
Unresolved items:
Next authorised action:
```

Do not say DRIFT FIXED until the correction is verified to the appropriate extent. Partial repairs remain ACTION REQUIRED with exact limitations.

4. Requirement traceability register

All statuses below reflect lack of implementation inspection, not a source-code audit.

IDs	Feature group	Current status	Evidence
VEL-001-003	Accounts, isolation, episodes	Planned / unverified	Requirements only
VEL-004-007	Private upload, draft, approval, patient view	Planned / unverified	Requirements only
VEL-008-010	Extraction, language, versioning	Planned / unverified	Requirements only
VEL-011-014	Caregivers, medicines/tasks, notifications	Planned / unverified	Requirements only
VEL-015-019	Check-ins, questions, dashboard, closure	Planned / unverified	Requirements only

IDs	Feature group	Current status	Evidence
VEL-020-021	Privacy/audit, metrics	Planned / unverified	Requirements only
Code 2	Tests & Follow-ups	Deferred by user	No implementation authorised yet

Split grouped rows into individual requirements as implementation begins. Evidence should name actual file paths, relevant test cases/commands and a commit or working-tree context. Avoid secrets and personal health data in this log.

5. Definition of done for a feature

- Linked to a requirement ID and current milestone.
- Expected user flow and errors work, not just the happy-path UI.
- Backend permission checks and clinic/patient scope enforced.
- State/version changes respect MASTER invariants.
- Relevant verification performed; remaining limitations explicit.
- Documentation/traceability updated to match actual behaviour.
- No unrelated user work removed; no hidden scope expansion.

6. Next-session checklist

1. Read these three files and applicable repository instructions.
2. Obtain the actual repository path/URL if missing.
3. Inspect what exists before scaffolding or overwriting anything.
4. Set up the first synthetic episode and expected manual care-plan output.
5. Choose auth/session approach and establish clinic/record access boundaries.
6. Derive M1 models and API contracts; then implement in small slices.
7. Explain feature flow and code responsibilities to Swayam.
8. Record changes and the next concrete task before ending.

7. Agent bootstrap instruction

Copy the following into the repository agent instructions when the repository is available, preserving any existing instructions:

Before implementing Velora features, read docs/VELORA_MASTER.md, docs/ARCHITECTURE.md and docs/SESSION_LOG.md. Keep Code 1 as active scope; Code 2 is deferred. When the user says Duck, execute the Duck protocol in SESSION_LOG.md against actual repository evidence. Correct ordinary in-scope drift, verify fixes and update the log. Do not claim completion without evidence or edit requirements merely to excuse implementation differences.

This bootstrap is an instruction for cooperating agents, not an automatic hook or background monitor. In a new environment, provide these files or repository access so the instruction can be followed.

8. Session history

2026-10-01 - Baseline documentation

- User requested three durable project documents and a Duck-triggered requirements/implementation comparison with autonomous in-scope correction.

- Created VELORA_MASTER.md, ARCHITECTURE.md and SESSION_LOG.md from the visible agreed conversation.
- Assigned requirement IDs VEL-001 through VEL-021 and separated M1, remaining Code 1, enhancements and deferred Code 2.
- Captured user journeys, roles, clinical approval, caregiver grants, document handling, versioning, reminders, check-ins, questions and operational boundaries.
- Architecture details are a proposed baseline; provider choices and clinical workflow remain open.
- No application code was inspected or modified; no application tests or deployment performed.
- Next: connect repository and begin M0/M1 as described above.

Future entry template

Date/time and timezone:
User request:
Repository / branch / commit / working-tree context:
Milestone and requirement IDs:
Files inspected:
Changes made:
Checks run and observed outcomes:
Duck verdict (if applicable):
Open decisions / blockers:
Next concrete task: